



Интро

- ТЕХНИКА

Проверка связи

Если у вас нет звука

убедитесь, что на устройстве и колонках включён звук
обновите страницу или переподключитесь к вебинару
откройте вебинар в другом браузере
перезагрузите устройство и войдите заново

Пожалуйста, отметьтесь в чате



если меня видно и слышно



если есть проблемы со звуком или видео

- 01 / Десктоп

С компьютера

Используйте **Google Chrome** или **Microsoft Edge**.

При проблемах - обновите страницу **F5**

- 02 / Мобильный

С телефона или планшета

Перейдите с мобильного интернета на **Wi-Fi**.

При проблемах - перезапустите приложение Webinar.

- 03 / Общий совет

Если ничего не помогло

Отключитесь и подключитесь заново.

Пишите в чат - организаторы помогут.

- ЗНАКОМСТВО

Перед началом

- КАКИЕ НОВОСТИ?

Что интересного произошло за неделю?

Напишите в чат - событие, новость или вывод из работы, учёбы или личного опыта.



- ПРЕПОДАВАТЕЛЬ КУРСА

Павлин Николай

Преподаватель кафедры бизнес-информатики МТУСИ
с 2021 года

СТО Nova · стартап в сфере мониторинга здоровья · резидент Сколково

Разрабатываю **LLM-приложения** и RAG-платформу **RAGMaster** (грант ФСИ)

Автор RAG-системы для охраны труда - **chat.kiout.ru**, 1000+ пользователей

Соавтор LLM, попавшей в **топ-20** русскоязычных моделей по MERA (02.2025)

7+ лет в коммерческой разработке: Python, FastAPI, LLM-стек, DevOps

Многократный победитель федеральных хакатонов



Telegram



YouTube



LinkedIn

- ДОГОВОРЁННОСТИ

Правила участия

- 1** Продолжительность вебинара - **80 минут**
- 2** Задавайте **вопросы в чате** - ответу в процессе или в конце
- 3** **Запись** будет доступна на платформе на следующий день
- 4** Обсуждение можно **продолжить в чате** после вебинара
- 5** **Воздерживаемся от обсуждения политических тем.** Здесь мы про разработку, ИИ и продукты - давайте сохраним фокус.

Начинаем!





Observability ИИ-приложений

Модуль 3 Блок 6

Павлин Н.К.

План занятия

Теория:

1. Зачем LLM нужна отдельная observability
2. Что лежит в ответе LLM: `usage`, `finish_reason`, кеш
3. Phoenix vs Langfuse vs LangSmith — что выбрать
4. PII в логах и трейсах: regex и Presidio
5. Базовые JSON-логи на `structlog`

Практика:

- Подключение Phoenix к FastAPI в `docker-compose`
- Маскирование email/телефона/карты перед логом

Что вы уже знаете

Из М1Б4 (теория стоимости и латентности):

- TTFT, throughput, Prompt Caching как концепция
- Расчёт стоимости по прайс-листу токенов

Из М3Б4 (FastAPI):

- Структура `routers` / `services` / `schemas` / `config`
- Middleware, DI для LLM-клиентов, async-вызовы

Из М3Б5 (Docker):

- `docker-compose up` поднимает app + Redis одной командой

Сегодня пристёгиваем к этому observability. В конце занятия Phoenix работает рядом с `app` в том же `docker-compose`.

Словарь занятия

Термин	English	Что это
Observability	Observability	Умение понять, что внутри системы, по внешним сигналам
Trace	Trace	Цепочка операций, связанных одним <code>trace_id</code>
Span	Span	Одна операция в трейсе с началом, концом и атрибутами
PII	Personally Identifiable Information	Персональные данные: email, телефон, карта, паспорт
Redaction	Redaction	Замена PII на плейсхолдер перед логированием

Зачем LLM нужна отдельная observability

Обычный HTTP-сервис: одинаковый ответ на одинаковый запрос, детерминизм, метрика — RPS и latency.

LLM-сервис:

- **Дорого:** один запрос стоит \$0.001–0.05, в сумме — тысячи долларов в месяц
- **Медленно:** p95 latency 3–30 секунд, не миллисекунды
- **Недетерминировано:** на одинаковый вход — разный ответ
- **Цепочки:** `prompt` → LLM → `tool_call` → `tool` → LLM → `ответ` — нужно видеть шаги
- **Качество:** ответ формально ОК, по сути — галлюцинация

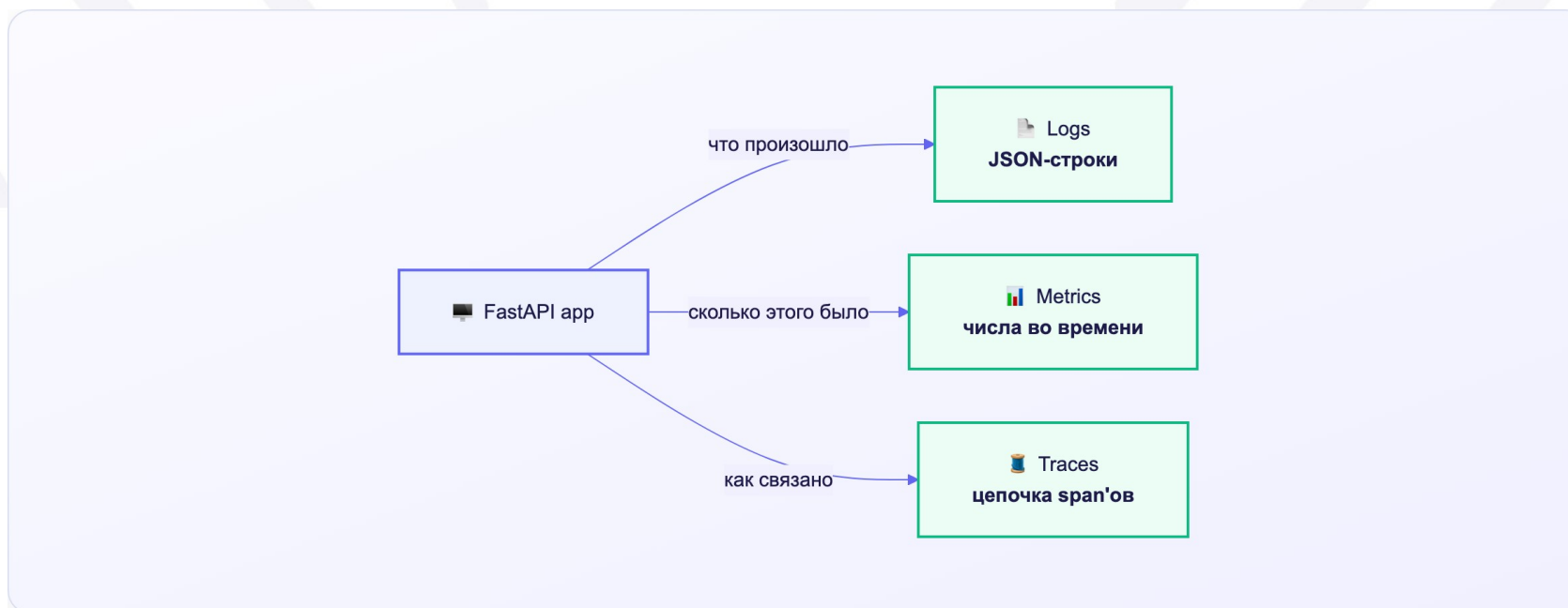
Отсюда специфичные вопросы:

- Какой промпт был отправлен и сколько токенов занял?
- Почему конкретно этот ответ дорогой?
- Почему модель выбрала не тот tool?
- Есть ли drift качества после смены модели?

Три столба observability

- **Logs** — дискретные события с богатым контекстом. «Что произошло в момент X с запросом Y».
- **Metrics** — числовые ряды. «Сколько таких событий за период».
- **Traces** — связанные события одного запроса. Waterfall: что за чем шло и сколько заняло.

Один **request_id** (он же **trace_id**) должен быть везде — тогда от лога вы попадаете в трейс одним кликом.



Что в ответе LLM: поля, на которые смотрим

example.python

```
1 response = client.chat.completions.create(  
2     model="gpt-5-mini",  
3     messages=[...],  
4 )  
5  
6 response.model                # "gpt-5-mini-2026-03-15" — snapshot  
7 response.choices[0].finish_reason # "stop" | "length" | "tool_calls" | "content_filter"  
8  
9 response.usage.prompt_tokens  # 487  
10 response.usage.completion_tokens # 203  
11 response.usage.total_tokens   # 690  
12  
13 response.usage.prompt_tokens_details.cached_tokens # 384
```

Что в ответе LLM: поля, на которые смотрим

На что смотрим:

- `finish_reason` — главный диагностический сигнал. `length` = промпт или `max_tokens` настроены плохо.
- `cached_tokens` — сколько входа пришло из кеша. На GPT-5 family это скидка 90% — без неё счёт улетает.
- `model` — конкретный snapshot. Меняется без предупреждения.

Не пиши своё — возьми готовое

Дальше — про инструменты.

Что было: разобрали, зачем нужна отдельная observability и какие сигналы есть в ответе LLM.

Что добавляем: какие готовые продукты для LLM observability существуют, как их сравнить и какой подключить за 15 минут.

LLM observability: ландшафт инструментов

Open-source, self-host, OTel-совместимы:

- [Arize Phoenix](#) — лёгкий старт, локальный запуск, vendor-agnostic
- [Langfuse](#) — self-hosted платформа, prompt management, scoring
- [OpenLLMetry / Traceloop](#) — единый OTel SDK для зоопарка LLM-SDK

Облачные / коммерческие:

- [LangSmith](#) — глубокая интеграция с LangChain/LangGraph
- **Datadog LLM Observability, Helicone, WhyLabs Langkit** — для больших организаций

Что объединяет: все принимают OpenTelemetry-span'ы со стандартными атрибутами `gen_ai.*`. Можно начать с Phoenix локально, потом мигрировать на Langfuse-cloud — код приложения почти не меняется.

Phoenix vs Langfuse vs LangSmith

Наш выбор для курса: Phoenix. Локальный, бесплатный, OTel-native, без регистрации.

Когда брать что-то другое:

- Уже на проекте LangChain/LangGraph → LangSmith
- Нужно версионирование промптов и оценщики → Langfuse

Критерий	Phoenix	Langfuse	LangSmith
Лицензия	Elastic (OSS)	MIT (OSS)	Закрытая
Self-host	Бесплатно	Бесплатно	Enterprise-only
Облако	Arize AX (платный)	Cloud free tier + Pro	Cloud-only
Прод-storage	Postgres	Postgres + ClickHouse	Cloud
Vendor-agnostic	Да, OTel-native	Да, OTel + свой SDK	Привязан к LangChain
Prompt management	Базовый	Сильный	Сильный
Evaluation	Встроенный	Scoring + datasets	Annotation queues
Порог входа	Минимальный	Средний	Минимальный (но облако)

Что показывает Phoenix UI

Total Traces15,292

Total Cost\$213.95663875

Latency P50⌚ 2.7s

Latency P99⌚ 43.5s

Stream🔌

Last Month📅

Spans

Traces

Sessions

Metrics

Config

Q filter condition (e.x. span_kind == 'LLM')

+

Root Spans

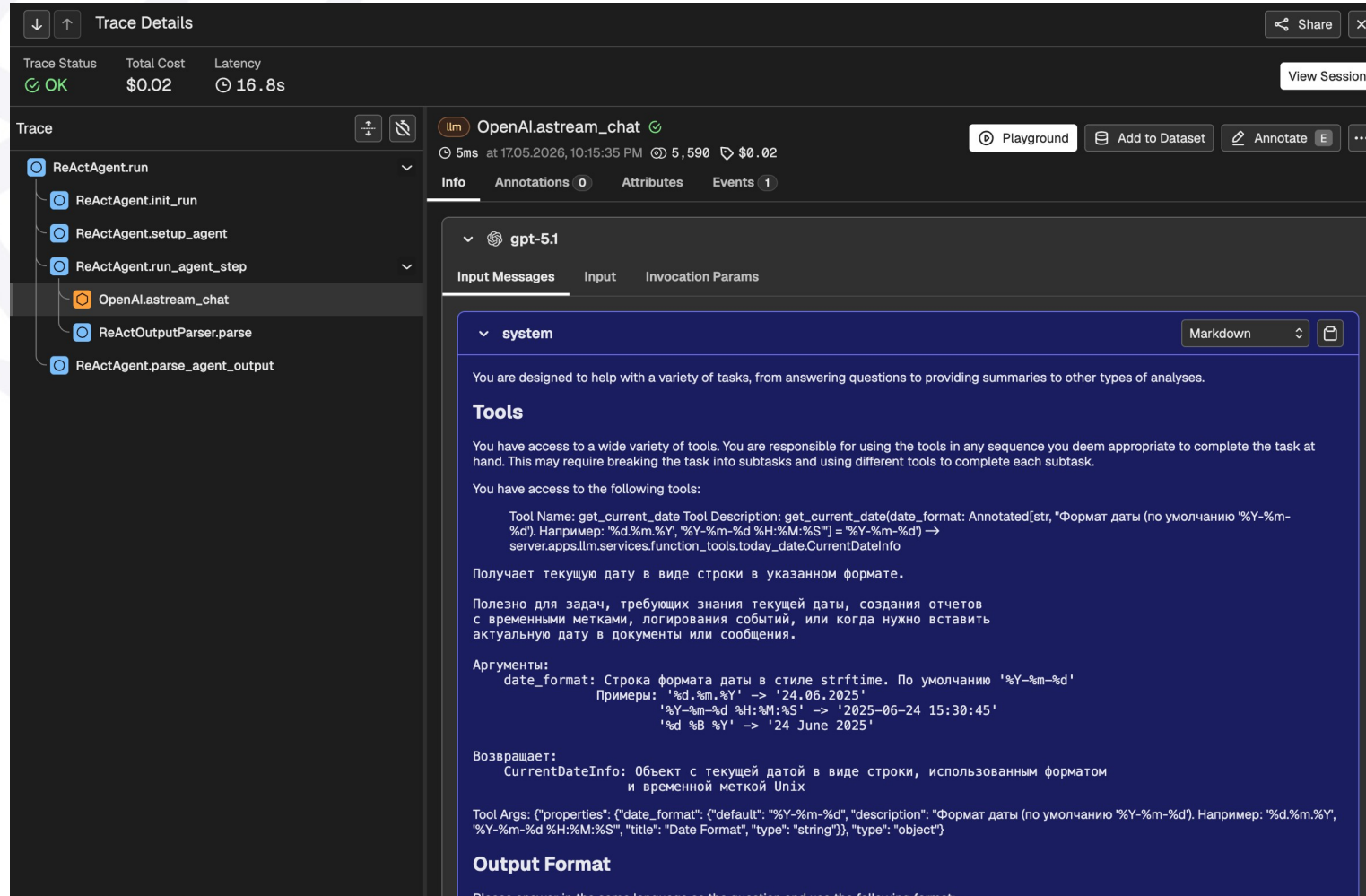
All

Columns⌵

⌵

status	kind	name	metadata	start time	latency	cumulative tokens	cumulative cost
✓	chain	ReActAgent.run	"chat_id":"d32372c5-2248...	17.05.2026, 10:30:54 ...	⌚ 9.1s	⌚ 6,036	💵 \$0.01
✓	chain	ReActAgent.run	"chat_id":"175dd03e-d02e...	17.05.2026, 10:30:38 ...	⌚ 28.8s	⌚ 49,073	💵 \$0.08
✓	chain	ReActAgent.run	--	17.05.2026, 10:20:06 ...	⌚ 1.2s	⌚ 535	💵 <\$0.01
✓	chain	ReActAgent.run	"chat_id":"d32372c5-2248...	17.05.2026, 10:15:35 P...	⌚ 16.8s	⌚ 5,590	💵 \$0.02
✓	chain	ReActAgent.run	"chat_id":"d32372c5-2248...	17.05.2026, 10:08:18 P...	⌚ 7.3s	⌚ 3,576	💵 \$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 10:05:06 ...	⌚ 1.7s	⌚ 568	💵 <\$0.01
✓	chain	ReActAgent.run	"chat_id":"d32372c5-2248...	17.05.2026, 10:03:56 ...	⌚ 13.2s	⌚ 2,951	💵 \$0.01
✓	llm	OpenAI.complete	--	17.05.2026, 09:58:23 ...	⌚ 1.6s	⌚ 173	💵 <\$0.01
✓	chain	ReActAgent.run	"chat_id":"d32372c5-2248...	17.05.2026, 09:58:20 ...	⌚ 6.5s	⌚ 1,967	💵 <\$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 09:58:20 ...	⌚ 2.6s	⌚ 540	💵 <\$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 09:50:06 ...	⌚ 3.2s	⌚ 538	💵 <\$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 09:35:06 ...	⌚ 2.5s	⌚ 538	💵 <\$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 09:20:06 ...	⌚ 3.3s	⌚ 535	💵 <\$0.01
✓	chain	ReActAgent.parse_agent_	--	17.05.2026, 09:05:10 ...	⌚ 1ms	⌚ 0	--
⚠	chain	ReActAgent.run	--	17.05.2026, 09:05:06 ...	⌚ 3.8s	⌚ 609	💵 <\$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 08:50:06 ...	⌚ 2.6s	⌚ 547	💵 <\$0.01
✓	chain	ReActAgent.run	--	17.05.2026, 08:35:06 ...	⌚ 2.6s	⌚ 556	💵 <\$0.01

Что показывает Phoenix UI



The screenshot displays the Phoenix UI interface, specifically the 'Trace Details' view. The top bar shows the trace status as 'OK', a total cost of '\$0.02', and a latency of '16.8s'. The trace is titled 'OpenAI.astream_chat' and includes a 'View Session' button. The left sidebar lists the trace steps: 'ReActAgent.run', 'ReActAgent.init_run', 'ReActAgent.setup_agent', 'ReActAgent.run_agent_step', 'OpenAI.astream_chat', 'ReActOutputParser.parse', and 'ReActAgent.parse_agent_output'. The main panel shows the 'gpt-5.1' model's input messages, including a system message that defines the agent's role and tools. The system message states: 'You are designed to help with a variety of tasks, from answering questions to providing summaries to other types of analyses. Tools You have access to a wide variety of tools. You are responsible for using the tools in any sequence you deem appropriate to complete the task at hand. This may require breaking the task into subtasks and using different tools to complete each subtask. You have access to the following tools: Tool Name: get_current_date Tool Description: get_current_date(date_format: Annotated[str, "Формат даты (по умолчанию '%Y-%m-%d')"]. Например: '%d.%m.%Y', '%Y-%m-%d %H:%M:%S') = '%Y-%m-%d' -> server.apps.llm.services.function_tools.today_date.CurrentDateInfo Получает текущую дату в виде строки в указанном формате. Полезно для задач, требующих знания текущей даты, создания отчетов с временными метками, логирования событий, или когда нужно вставить актуальную дату в документы или сообщения. Аргументы: date_format: Строка формата даты в стиле strftime. По умолчанию '%Y-%m-%d' Примеры: '%d.%m.%Y' -> '24.06.2025' '%Y-%m-%d %H:%M:%S' -> '2025-06-24 15:30:45' '%d %B %Y' -> '24 June 2025' Возвращает: CurrentDateInfo: Объект с текущей датой в виде строки, использованным форматом и временной меткой Unix Tool Args: {'properties': {'date_format': {'default': '%Y-%m-%d', 'description': 'Формат даты (по умолчанию '%Y-%m-%d')'. Например: '%d.%m.%Y', '%Y-%m-%d %H:%M:%S', 'title': 'Date Format', 'type': 'string'}), 'type': 'object'}}

Phoenix: подключение

Самый короткий путь — три строки:

```
example.python

1 # pip install arize-phoenix openinference-instrumentation-openai
2 from phoenix.otel import register
3 from openinference.instrumentation.openai import OpenAIInstrumentor
4
5 tracer_provider = register(project_name="example",
6                             endpoint="http://phoenix:6006")
7 OpenAIInstrumentor().instrument(tracer_provider=tracer_provider)
8
9 # Далее любой client.chat.completions.create автоматически создаёт span
10 # с input messages, output, tokens, latency, finish_reason.
```

Phoenix: подключение

В дипломном проекте Phoenix идёт как сервис в **docker-compose**:

```
example.yaml
1  services:
2    phoenix:
3      image: arizephoenix/phoenix:latest
4      ports:
5        - "6006:6006" # UI
6        - "4317:4317" # OTLP gRPC
7      environment:
8        PHOENIX_WORKING_DIR: /data
9      volumes:
10       - phoenix-data:/data
11  app:
12    environment:
13      PHOENIX_COLLECTOR_ENDPOINT: http://phoenix:6006
14    depends_on:
15      - phoenix
16  volumes:
17    phoenix-data:
```

Phoenix: подключение

После `docker-compose up` UI открывается на <http://localhost:6006>.

Безопасность данных в логах и трейсах

Phoenix и Langfuse удобны, но создают риск: всё, что туда попало, становится отдельным хранилищем персональных данных.

Что было: подключили готовый observability-инструмент.

Что добавляем: PII-маскирование regex и Microsoft Presidio, retention policy, ответ на вопрос «где маскировать».

PII: что это и почему важно

PII (Personally Identifiable Information) — данные, по которым можно идентифицировать человека.

Типовые категории:

- Прямые: email, телефон, паспорт, СНИЛС, ИНН
- Финансовые: номер карты (PAN), CVV, IBAN, счёт
- Технические: IP, cookie, device_id
- Косвенные: ФИО + город + дата рождения (квази-идентификатор)

Почему не держим PII в логах и трейсах:

- **Регулятор:** 152-ФЗ в РФ, GDPR в ЕС — персональные данные хранятся только с согласием и целью
- **Доступ:** логи и Phoenix UI обычно открыты всей команде, sidecar-агентам, backup-системам
- **Вендор-риск:** если Phoenix/Langfuse в облаке — это передача данных третьим лицам

Правило дипломного проекта: сырой промпт в логах — никогда. Только хеш + маскированный превью.

PII: regex как MVP

```
example.python

1  import re, hashlib
2
3  PII_PATTERNS = {
4      "EMAIL": re.compile(r"\b[\w.+~]+@[ \w.-]+\.[A-Za-z]{2,}\b"),
5      "PHONE_RU": re.compile(r"\b(?:\+7|8)[\s-]?(?:\d{3})?[\s-]?"
6                           r"\d{3}[\s-]? \d{2}[\s-]? \d{2}\b"),
7      "CARD": re.compile(r"\b(?:\d{4}[\s-]?){3}\d{4}\b"),
8      "INN": re.compile(r"\b(?:\d{10}|\d{12})\b"),
9      "PASSPORT": re.compile(r"\b\d{2}\s?\d{2}\s?\d{6}\b"),
10 }
11
12 def redact_pii(text: str) -> str:
13     for name, pattern in PII_PATTERNS.items():
14         text = pattern.sub(f"[{name}]", text)
15     return text
```

PII: regex как MVP

example.python

```
1 def prompt_hash(text: str) -> str:
2     return "sha256:" + hashlib.sha256(text.encode()).hexdigest()[ :16]
3
4 # Использование перед логированием:
5 raw = "Мой email ivan@mail.ru, тел +7 (999) 123-45-67, карта 4111 1111 1111 1111"
6 logger.info("llm_request",
7             prompt_hash=prompt_hash(raw),
8             prompt_preview=redact_pii(raw)[ :120])
9 # prompt_preview: "Мой email [EMAIL], тел [PHONE_RU], карта [CARD]"
```

PII: Microsoft Presidio

example.python

```
1 # pip install presidio-analyzer presidio-anonymizer
2 # python -m spacy download ru_core_news_md
3 from presidio_analyzer import AnalyzerEngine
4 from presidio_anonymizer import AnonymizerEngine
5 from presidio_anonymizer.entities import OperatorConfig
6
7 analyzer = AnalyzerEngine(supported_languages=["en", "ru"])
8 anonymizer = AnonymizerEngine()
9
10 def redact_with_presidio(text: str, language: str = "ru") -> str:
11     results = analyzer.analyze(
12         text=text,
13         language=language,
14         entities=["EMAIL_ADDRESS", "PHONE_NUMBER", "CREDIT_CARD",
15                 "PERSON", "LOCATION", "IP_ADDRESS"],
16     )
17     return anonymizer.anonymize(
18         text=text,
19         analyzer_results=results,
20         operators={"DEFAULT": OperatorConfig("replace", {"new_value": "[PII]"}),
21     ).text
22
23 # "Иван Петров, ivan@mail.ru" -> "[PII], [PII]"
```

PII: Microsoft Presidio

Когда Presidio выигрывает у regex:

- Имена, адреса, организации (NER через spaCy)
- Контекстные правила (рядом со словом «паспорт»)
- Свои recognizers под домен (номер полиса, медкарта)

Retention: сколько хранить

Правила:

- Удаление автоматическое, не по запросу
- Срок указан в политике конфиденциальности
- Запрос «удалите мои данные» (152-ФЗ, GDPR) исполним за разумный срок

Что	Срок	Где	Почему
Сырые промпты (полные)	1–7 дней	secure storage с шифрованием	Расследование инцидентов
PII-маскированные логи	30 дней	structured logs	Аналитика, поиск аномалий
Трейсы Phoenix	3–14 дней	Phoenix backend	Performance-отладка
Датасеты для evaluation	постоянно	отдельно, с согласием	Б3.7

Базовые JSON-логи: structlog в двух строках

Когда Phoenix есть, отдельный логгер всё ещё полезен — для HTTP-уровня (request_id, status, latency) и аудита.

```
example.python

1  # pip install structlog
2  import structlog
3
4  structlog.configure(
5      processors=[
6          structlog.contextvars.merge_contextvars,
7          structlog.processors.add_log_level,
8          structlog.processors.TimeStamper(fmt="iso", utc=True),
9          structlog.processors.JSONRenderer(ensure_ascii=False),
10     ],
11 )
12
13 logger = structlog.get_logger()
14 logger.info("llm_request_completed",
15     request_id="a1b2c3d4",
16     model="gpt-5-mini",
17     input_tokens=487,
18     output_tokens=203,
19     latency_ms=5243,
20     finish_reason="stop")
```

Базовые JSON-логи: structlog в двух строках

Что критично:

- `JSONRenderer(ensure_ascii=False)` — поддержка кириллицы
- `contextvars.merge_contextvars` — позволяет привязать `request_id` в middleware один раз и забыть
- Уровень — через env var: в деве DEBUG, в проде INFO

Антипаттерны observability

- **«Логируем всё подряд»** — логи раздуваются, важное тонет в шуме
- **Полный prompt в обычный лог** — PII-утечка, комплаенс-проблема
- **Нет request_id** — невозможно сшить цепочку, каждый лог — островок
- **Phoenix поставили, но никто не смотрит** — трейсы без наблюдателя мертвы
- **Алерты без action plan** — будят ночью и не говорят, что делать
- **High-cardinality labels в метриках (user_id, prompt_hash)** — счётчики взрываются
- **Смешанные таймзоны в логах** — невозможно восстановить хронологию
- **Своя реализация трейсинга** — год работы, экспертизы которой нет

Правило: observability — это инвестиция, которая окупается через 1–2 инцидента. Если вложились, но никто не пользуется — пересмотрите процесс.

Итоги занятия

Теперь вы умеете:

- Объяснять, чем LLM-сервис отличается от обычного HTTP по части наблюдаемости
- Читать `usage` в ответе OpenAI: токены, кеш, `finish_reason`
- Подключать Phoenix к FastAPI за 15 минут (одним docker-сервисом)
- Сравнивать Phoenix / Langfuse / LangSmith и выбирать под задачу
- Маскировать PII через regex и Presidio перед логом и трейсом
- Писать базовые JSON-логи на `structlog` с `request_id`

Культурно:

- Свой logging+metrics+tracing не пишем — берём готовое
- Observability — не «логирование», это ответы на производственные вопросы
- Без PII-стратегии Phoenix становится хранилищем персональных данных

ДЗ-самост: подключить Phoenix к FastAPI

Что сделать в своём FastAPI-сервисе (Б3.4 + Б3.5):

1. **Phoenix в docker-compose** — сервис рядом с `app`, UI на `:6006`
2. **Автоинструментация** — `OpenAIInstrumentor().instrument(...)` на старте приложения
3. **structlog** — JSON-формат с `request_id`, `model`, `input_tokens`, `output_tokens`, `latency_ms`, `finish_reason`
4. **PII-маскирование** — regex для email/phone/card; `prompt_hash` + `prompt_preview` в логе

Референсы

Инструменты observability:

- [Arize Phoenix](#) и [Phoenix Docs](#) — OSS LLM observability
- [Langfuse](#) — OSS альтернатива с prompt management
- [LangSmith Docs](#) — для LangChain/LangGraph
- [OpenLLMetry / Traceloop](#) — единый OTel SDK
- [Microsoft Presidio](#) — PII detection
- [structlog](#) — structured logging

Стандарты:

- [OpenTelemetry GenAI Semantic Conventions](#) — атрибуты `gen_ai.system`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`
- [OpenTelemetry Python SDK](#)

API-документация:

- [OpenAI API Reference — Chat Completion object](#) — `usage`, `prompt_tokens_details`
- [OpenAI Prompt Caching](#) — условия и `cached_tokens`